ÖVNINGSUPPGIFTER: LEKTION 3

Steg-för-steg-normalisering (M:M), Lucidchart-härdning och slank Mockaroo-import

STARTPUNKT: Förbered din miljö!

1. Ladda ner och exekvera **Startskriptet för Lektion 3** från studentportalen (det rensar och skapar databasen `FailSafe_Lektion3`).
2. Kör därefter följande initiala kodblock i SSMS för att initiera kaostabellen innan lärarens code-along startar:

sql

```sql
USE FailSafe_Lektion3;
GO

CREATE TABLE Legacy_Incident_Chaos (
    ServerName NVARCHAR(100),
    Incident_And_Date NVARCHAR(250),
    InvestigatedByTechs NVARCHAR(MAX)
);

INSERT INTO Legacy_Incident_Chaos VALUES
('Prod-Web-01', 'DDoS:2026-11-05', 'Erik Nilsson, Lina Berg'),
('Prod-Web-01', 'SQLi-Attempt:2026-11-06', 'Lina Berg'),
('Dev-Db-01', 'UnauthorizedAccess:2026-11-07', 'Erik Nilsson, Anna Stark'),
('Test-App-04', 'None:2026-11-08', 'Kalle Blomqvist');
GO
```

GRÖN NIVÅ: Grundläggande normalisering

Uppgift 1: Atomär-detektiven

- **Beskrivning:**
  Analysera strukturen i den ärvda platta tabellen `Legacy_Incident_Chaos`.
- **Krav:**
  Skriv en SQL-kommentar i ditt skript som namnger exakt vilka två kolumner som förhindrar tabellen från att nå 1NF, samt en kort motivering baserad på begreppet "atomära värden".
- **Verifiering:**
  Kontrollera att din motivering belyser faran med sammansatta strängar och kommaseparerade listor.

Uppgift 2: Upprättandet av 1NF

- **Beskrivning:**
  Implementera en tabellstruktur som uppfyller kraven för 1NF och atomisera rådata.
- **Krav:**
  Skapa tabellen `Incident_1NF` med en fungerande `IDENTITY(1,1)` som Primary Key. Skriv en `INSERT INTO`-sats som flyttar och splittrar raden för `'Prod-Web-01'` (incidenten `2026-11-05`) över två unika rader.
- **Verifiering:**
  Kör `SELECT * FROM Incident_1NF` och verifiera att incidenten har brutits upp på två rader med varsin unik tekniker.

Uppgift 3: Brytningen till 2NF

- **Beskrivning:**
  Eliminera partiella beroenden genom att separera data baserat på livscykler.
- **Krav:**
  Skriv en förklaring i en SQL-kommentar som beskriver varför kolumnen `ServerName` dupliceras i onödan i 1NF, samt hur uppdelningen i tabellerna `IncidentServers_2NF` och `Incidents_And_Techs_2NF` löser detta.
- **Verifiering:**
  Kontrollera att din kommentar tydligt definierar vad ett "partiellt beroende" är.

🟡 GUL NIVÅ: Avancerad städning & M:M-kopplinger

Uppgift 4: Det transitiva giftet

- **Beskrivning:**
  Idenitfiera och isolera transitiva beroenden för att lyfta arkitekturen till 3NF.
- **Krav:**
  Granska tabellen `Incidents_And_Techs_2NF`. Identifiera vilken icke-nyckelkolumn som är transitivt beroende av en annan icke-nyckelkolumn. Skriv en kommentar om hur detta kan leda till en *Update Anomaly*.
- **Verifiering:**
  Säkerställ att du har identifierat teknikerns namn som det dolda beroendet.

Uppgift 5: Kopplingsbygget

- **Beskrivning:**
  Implementera en ren kopplingstabell för att lösa upp en Many-to-Many (M:M) relation.
- **Krav:**
  Skriv DDL-koden för att skapa kopplingstabellen `IncidentAssignments`. Tabellen måste länka samman incidenter med tekniker och använda en sammansatt primärnyckel.
- **Exempel:**
  `CONSTRAINT PK_IncidentAssignments PRIMARY KEY (IncidentId, TechId)`

- **Verifiering:**
  Verifiera i SSMS Object Explorer att tabellen har en sammansatt nyckel (två nyckel-ikoner).

Uppgift 6: M:M-Verifieringen

- **Beskrivning:**
  Skriv en sammansatt SQL-fråga som validerar att 3NF-strukturen kan sammanställas felfritt.
- **Krav:**
  Skriv en `INNER JOIN`-fråga som kopplar samman `Incidents`, `IncidentAssignments` och `SecurityTechs`.
- **Verifiering:**
  Kör frågan och kontrollera att relationerna stämmer överens med den ursprungliga rådatan.

🔴 RÖD NIVÅ: Arkitekturhärdning & Datasimulering

Uppgift 7: Greenfield M:M-Härdning (Från User Stories till DDL)

- **Beskrivning:**
  Översätt och härda en helt ny Greenfield M:M-arkitektur baserad på User Stories från produktägaren till säker, produktionsredo SQL-kod.
- **Kundens User Stories (Kravunderlag):**
  - **User Story 1**: Som säkerhetschef vill jag kunna definiera backupscheman (PolicyName) med ett tidsintervall (IntervalHours).
  - **User Story 2**: Som infrastrukturtekniker vill jag kunna registrera servrar (ServerName).
  - **User Story 3**: Som DevSecOps-ingenjör vill jag kunna koppla flera backupscheman till en server, och vice versa.
- **Krav:**
  Utgå från User Stories ovan (Backupscheman med tidsintervall, Registrera servrar samt koppla flera backupscheman till en server och vice versa). Rita upp modellen i Lucidchart med tre tabeller: `BackupServers`, `BackupPolicies` och kopplingstabellen `ServerPolicyMaps`. Exportera koden till SSMS och **härda den manuellt**: Lägg till `IDENTITY(1,1)` på dina PK-heltal, sätt `NOT NULL` och applicera en sammansatt primärnyckel på kopplingstabellen.
- **Verifiering:**
  Exekvera DDL-koden i SSMS och säkerställ att alla tre tabellerna skapas utan syntaxfel.

Uppgift 8: Unikhetsspärren

- **Beskrivning:**
  Säkra databasen mot logiska dubbletter på de oberoende tabellerna.
- **Krav:**
  Lägg till en `UNIQUE`-constraint på kolumnen `PolicyName` i tabellen `BackupPolicies`. Skriv två identiska `INSERT`-satser med policynamnet `'Daily-Incremental'` för att testa spärren.
- **Verifiering:**
  Det andra `INSERT`-försöket **måste** misslyckas och kasta ett *Violation of UNIQUE KEY constraint*-fel i SSMS.

Uppgift 9: Det slanka Mockaroo-schemat

- **Beskrivning:**
  Konfigurera en automatiserad datasimulering för en kopplingstabell via Mockaroo.
- **Krav:**
  Gå till Mockaroo. Konfigurera ett slankt schema för kopplingstabellen `ServerPolicyMaps`. Sätt `ServerId` till heltal (Integer) mellan `1` och `3`, och `PolicyId` till heltal mellan `1` och `2`. Ladda ner som SQL.
- **Verifiering:**
  Kontrollera i SQL-filen att inga genererade ID-värden faller utanför de angivna intervallen.

🚀 EXTRA UTMANINGAR: Pipeline-idempotens & Utbyggnad

Uppgift 10: Den säkra trestegsraketen

- **Beskrivning:**
  Montera ihop ett fullständigt, idempotent och automatiserat databasskript för din nya M:M-modell.
- **Krav:**
  Sätt ihop rensningsblocket från det officiella startupskriptet (`IF EXISTS... DROP DATABASE`), din manuellt härdade DDL från Uppgift 7/8, och din nedladdade Mockaroo-DML från Uppgift 9 till en enda fil.
- **Verifiering:**
  Exekvera hela skriptet (F5) tre gånger i rad. Det måste köras felfritt varje gång.

Uppgift 11: Sammansatt Ändringslogg (Enterprise-utbyggnad)

- **Beskrivning:**
  Bygg ut det befintliga skriptet med en avancerad ändringslogg (`Audit Trail`) som refererar direkt till kopplingstabellens sammansatta nyckel.
- **Krav:**
  Skapa tabellen `BackupAuditTrail`. Denna tabell måste ha en **sammansatt Foreign**

Key (två kolumner) som pekar på `ServerPolicyMaps(ServerId, PolicyId)`. Lägg till automatiskt användar- och tidsspår via `DEFAULT SYSTEM_USER` och `DEFAULT GETDATE()`.

- **Verifiering:**
  Kontrollera att tabellen skapas och att den sammansatta främmande nyckeln (Composite FK) valideras korrekt i Object Explorer.
  Uppgift 12: Pipeline-verifieraren

- **Beskrivning:**
  Skapa ett automatiserat verifieringsblock med `UNION ALL` och `COUNT(*)` för att logga radantalet för alla ingående tabeller i din nya enterprise-modell.
- **Verifiering:**
  Kör skriptet och kontrollera att alla tabeller (inklusive kopplingstabellen och ändringsloggen) spottar ut korrekta rader i resultatet.